



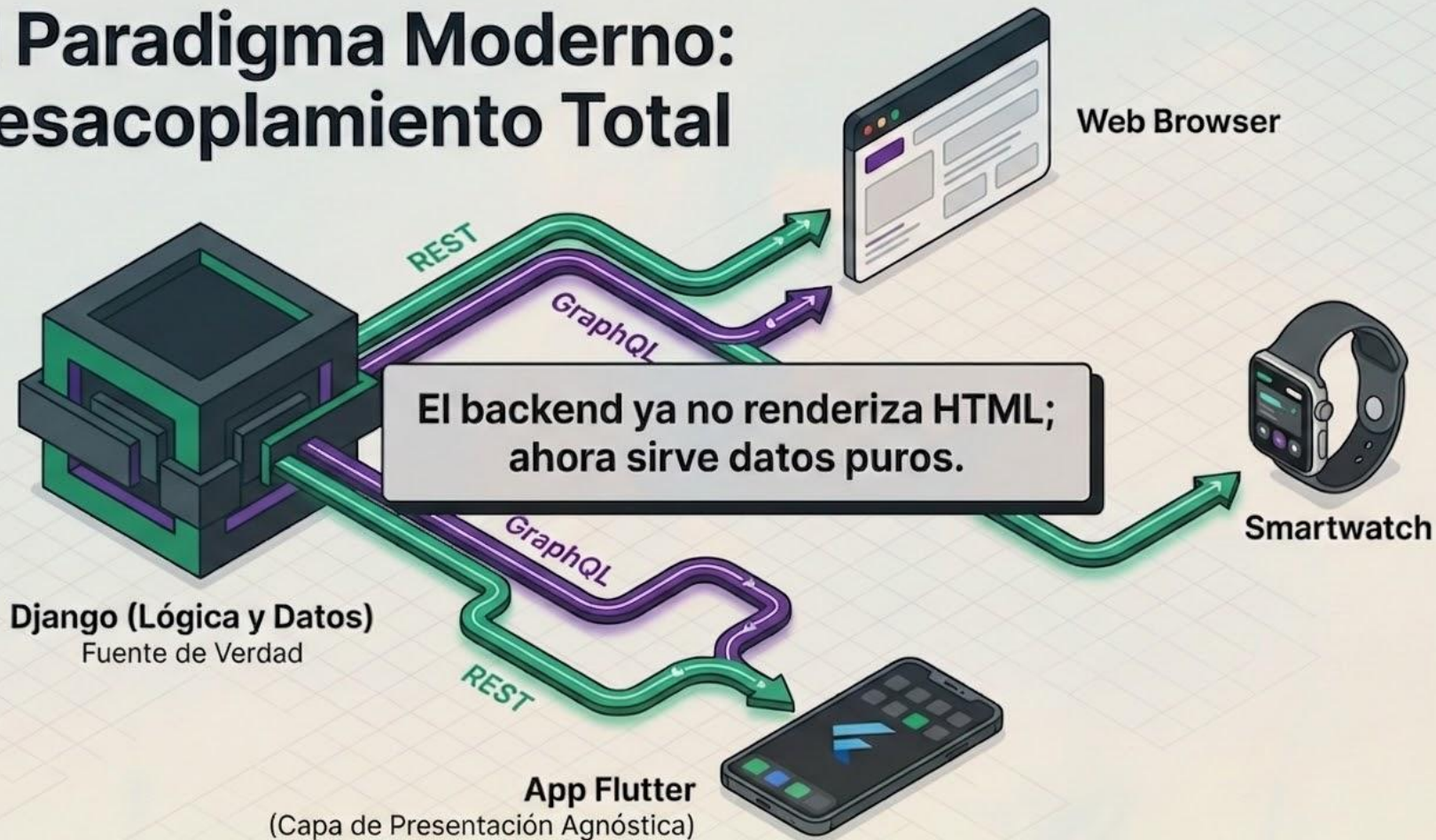
APIs

DRF vs GraphQL

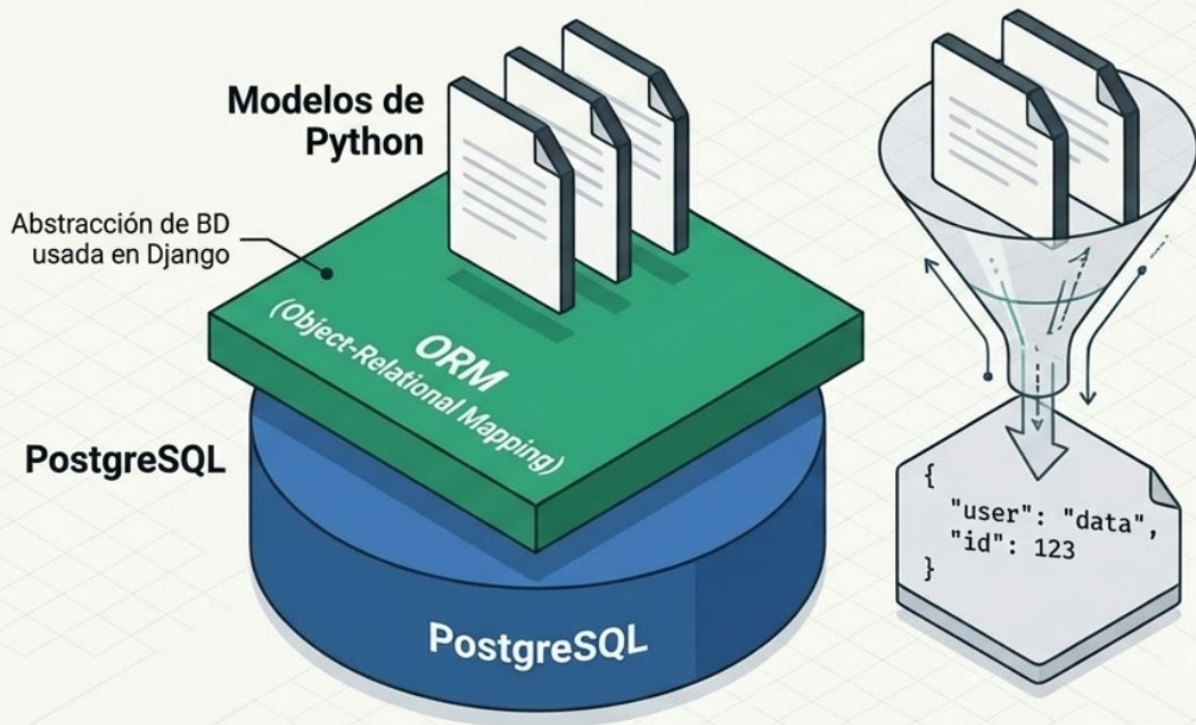
...

by M.T.I. Juan Luis Campos Barrera

El Paradigma Moderno: Desacoplamiento Total



El Cimiento: Django y el Desarrollo Rápido



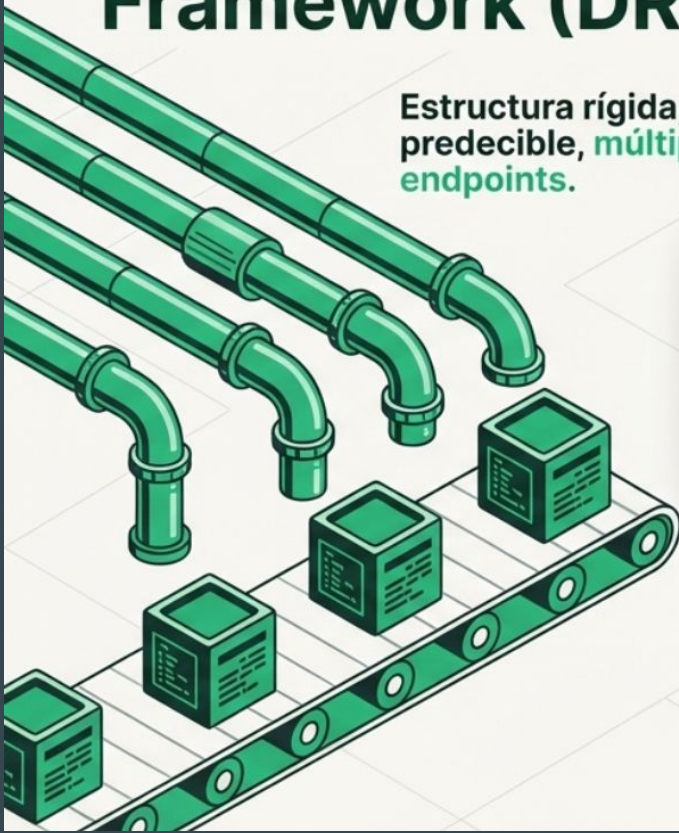
Objetivo: Rapid Application Development para MVPs y lógica de negocio compleja.

Mecanismo: El ORM traduce tablas relacionales en clases de Python de alto nivel.

El Reto:
¿Cómo estructuramos y entregamos estos modelos a la capa de presentación?

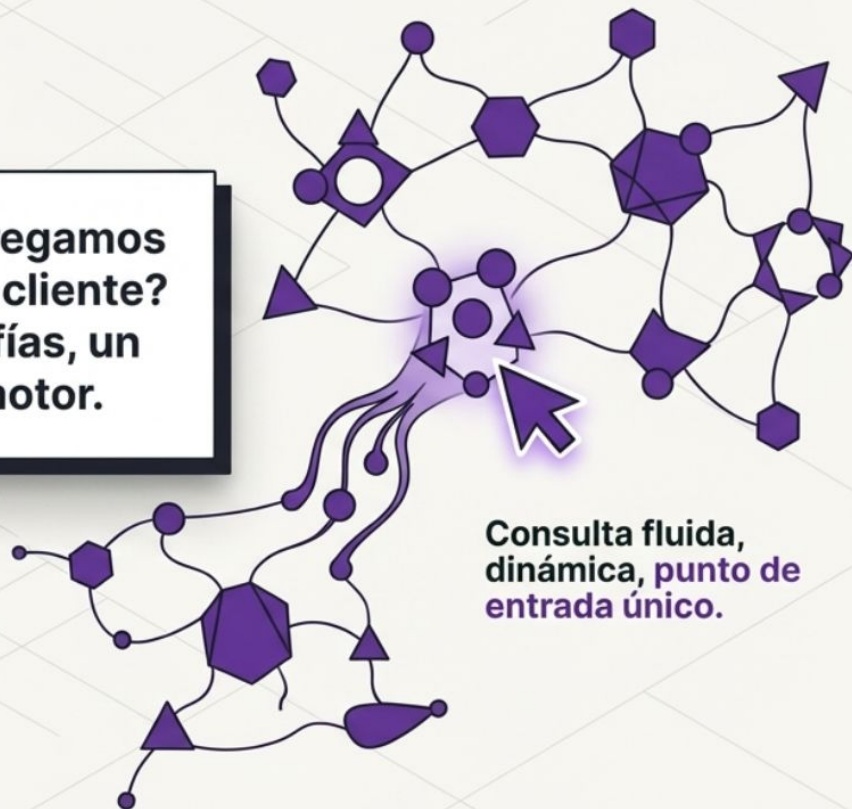
Django REST Framework (DRF)

Estructura rígida,
predecible, **múltiples**
endpoints.



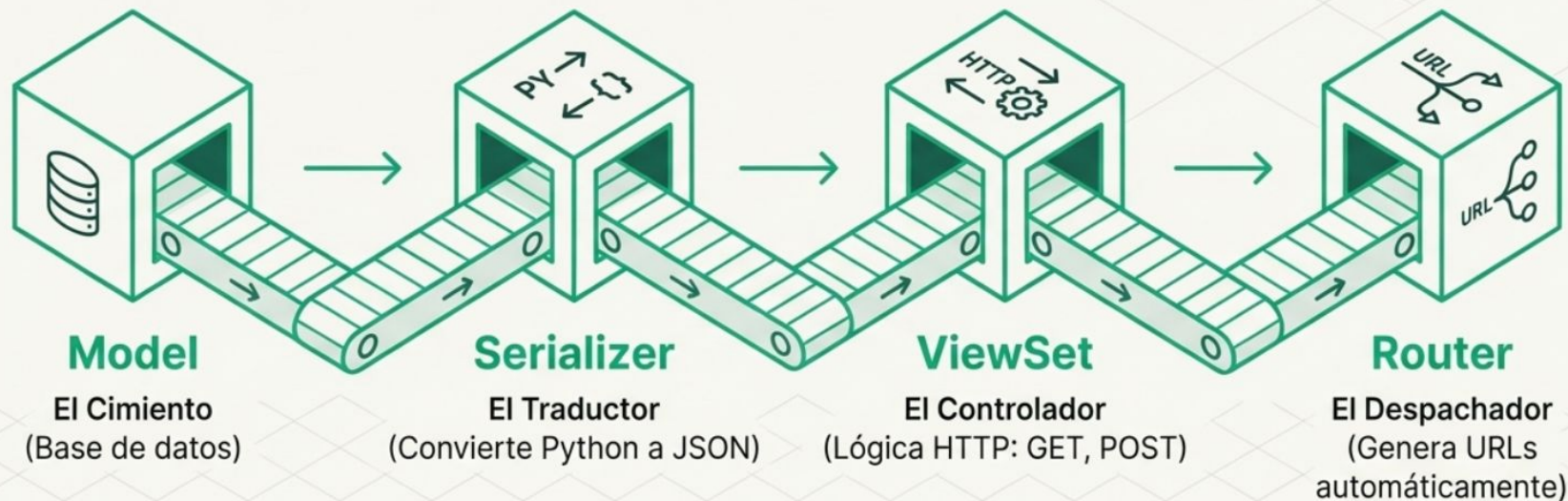
¿Cómo entregamos
los datos al cliente?
Dos filosofías, un
mismo motor.

GraphQL



Consulta fluida,
dinámica, **punto de**
entrada único.

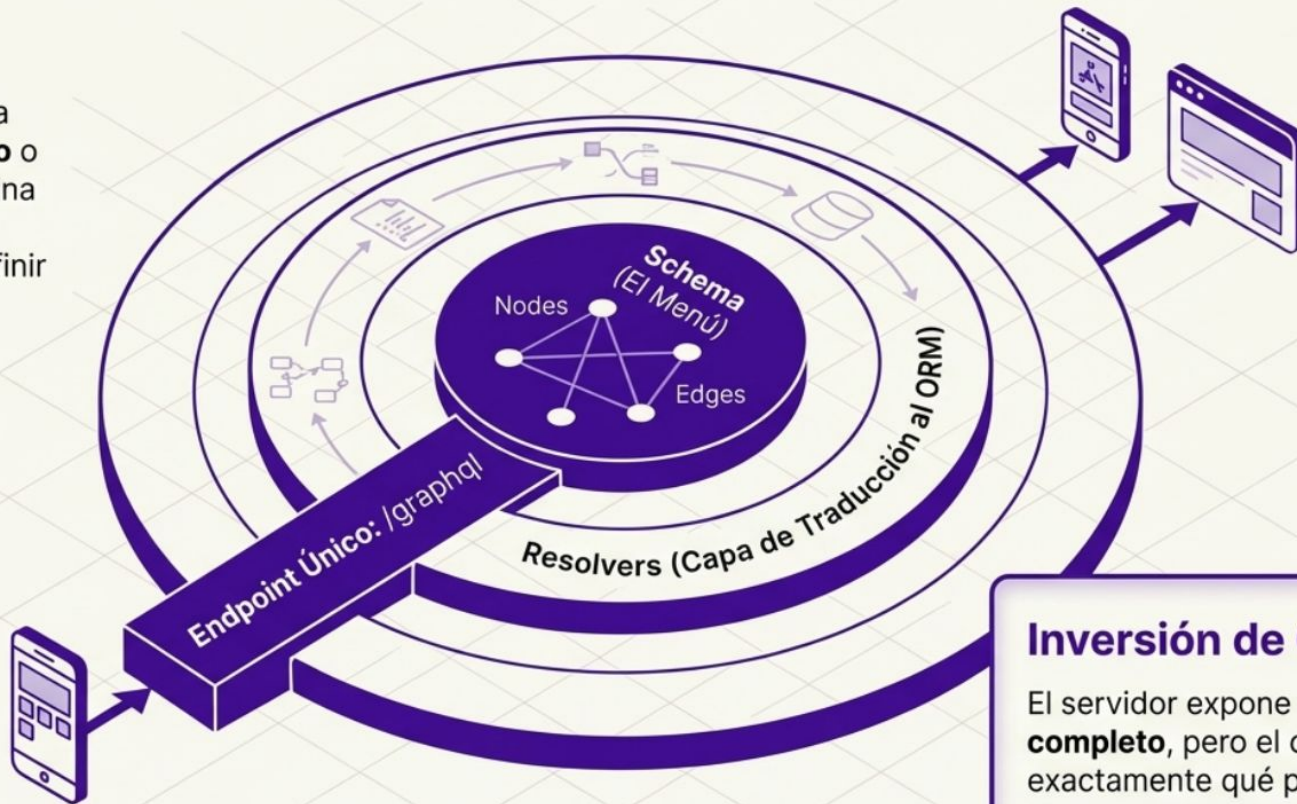
Anatomía de Django REST Framework



DRF automatiza la creación de APIs RESTful. Conecta modelos de datos directamente con rutas estandarizadas, promoviendo convenciones estrictas sobre configuración.

Anatomía de GraphQL en Django

Implementado vía **graphene-django** o **strawberry**. Elimina por completo la necesidad de definir múltiples URLs.



Inversión de Control

El servidor expone un **"menú" completo**, pero el cliente decide exactamente qué platos pedir.

Matriz de Diagnóstico Arquitectónico

Atributo	REST Framework	GraphQL
Filosofía de Rutas	Múltiples endpoints estructurados (/repartidor, /entregas)	Un solo endpoint unificado (/graphql)
Control de Datos	El servidor dicta la forma de la respuesta	El cliente modela la respuesta deseada
Curva de Aprendizaje	Baja/Media (Estándar de industria)	Alta (Sintaxis y tipado estricto)
Caché HTTP Nativo	Excelente (A nivel de URL)	Complejo (Requiere herramientas de cliente)

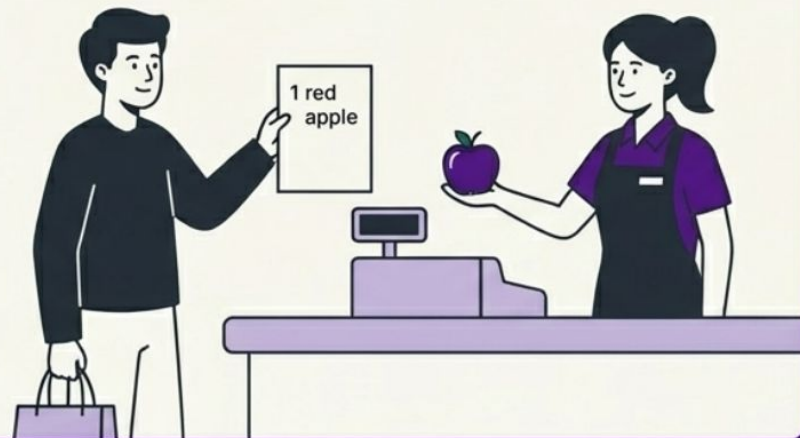
El Problema de la Transferencia de Datos

Over-fetching (REST)



Over-fetching (REST): El cliente descarga KB de datos innecesarios porque el Serializer es estático. Requiere múltiples viajes para vistas complejas (Under-fetching).

Exact Fetching (GraphQL)



La Solución GraphQL: Transferencia quirúrgica. Cero desperdicio de ancho de banda. El cliente recibe exactamente lo que pidió.

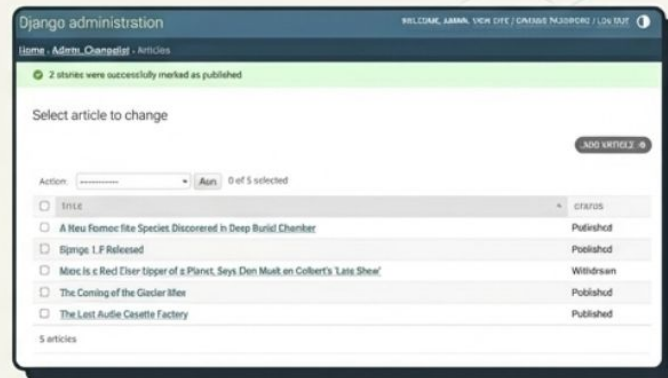
Escenario Práctico: Sistema de Logística Universitaria



Flutter App (Repartidor)

Mission-Briefing

- **Componente 1:** Core Administrativo en Django.
- **Componente 2:** App Móvil de Repartidores en Flutter.



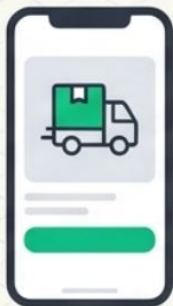
Core Backend (Django Admin)

El Requerimiento: La pantalla principal de la app requiere mostrar el Perfil del Repartidor y su Lista de Entregas Pendientes.

La Pregunta Central: ¿Cómo resolvemos el flujo de datos para esta pantalla usando **REST** vs. **GraphQL**?

La Solución REST: Diseño del Backend

En DRF, la estandarización nos obliga a consultar dos recursos separados.



```
GET /api/repartidores/123/
```

Retorna el RepartidorSerializer



```
GET /api/entregas/?repartidor=123&estado=pendiente
```

Retorna el EntregaSerializer



Latencia duplicada: Dos viajes de red (round-trips).

La Solución **REST**: El Payload

payload.json

```
{
  "id": 123,
  "nombre": "Carlos",
  "fecha_registro": "2021-04-12T08:00:00Z",
  "ultimo_login": "2023-10-01T09:30:00Z",
  "entregas": [
    {
      "id_paquete": "PKG-992",
      "direccion": "Edificio A, Laboratorio 3",
      "peso_kg": 2.5,
      "estado": "pendiente"
    }
  ]
}
```

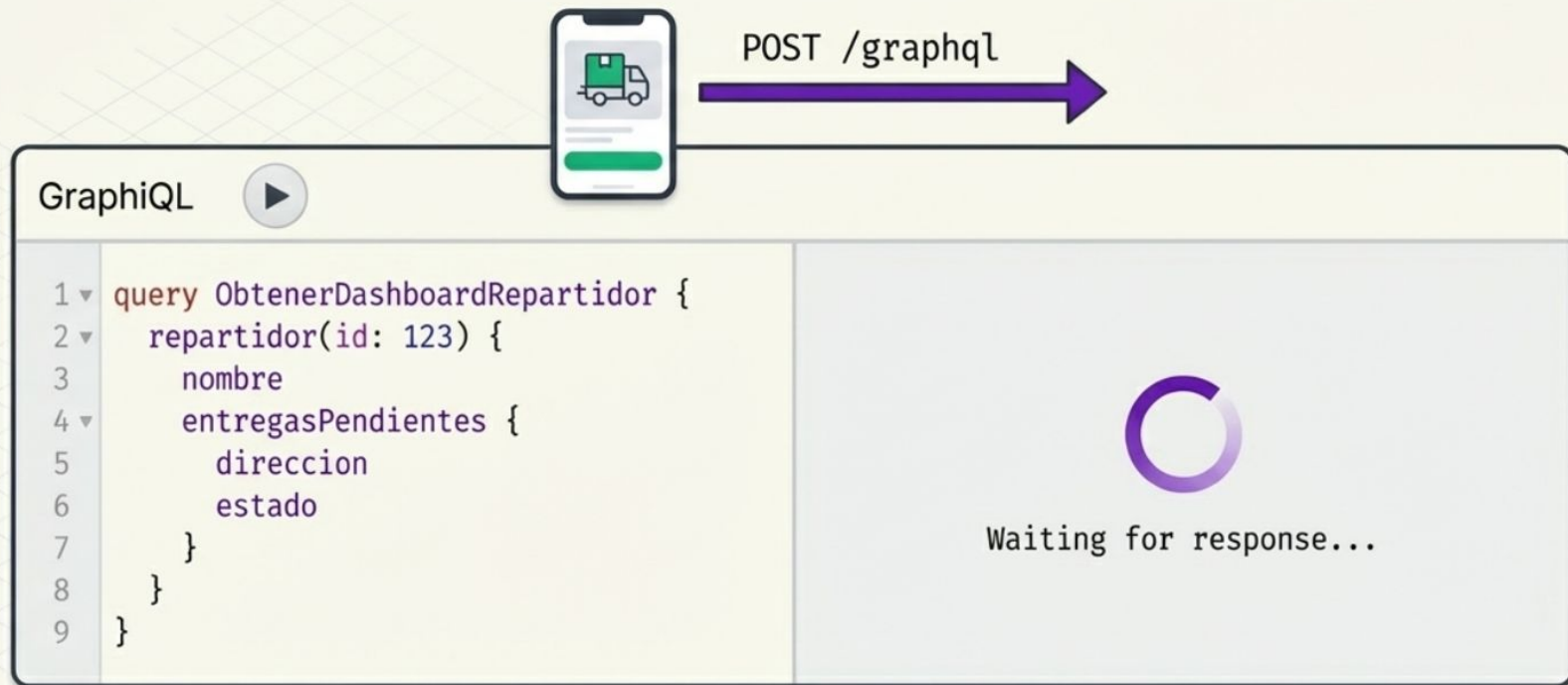
⚠ OVER-FETCHING

⚠ OVER-FETCHING

⚠ OVER-FETCHING

El Serializer devuelve toda la estructura del modelo. La app móvil consume batería y ancho de banda procesando datos que la interfaz de usuario no va a renderizar.

La Solución GraphQL: La Consulta del Cliente



Flutter envía una sola petición POST al endpoint unificado.
La consulta define explícitamente la forma esperada.



Eficiencia máxima:
Un solo viaje por la red.

La Solución GraphQL: El Payload Exacto

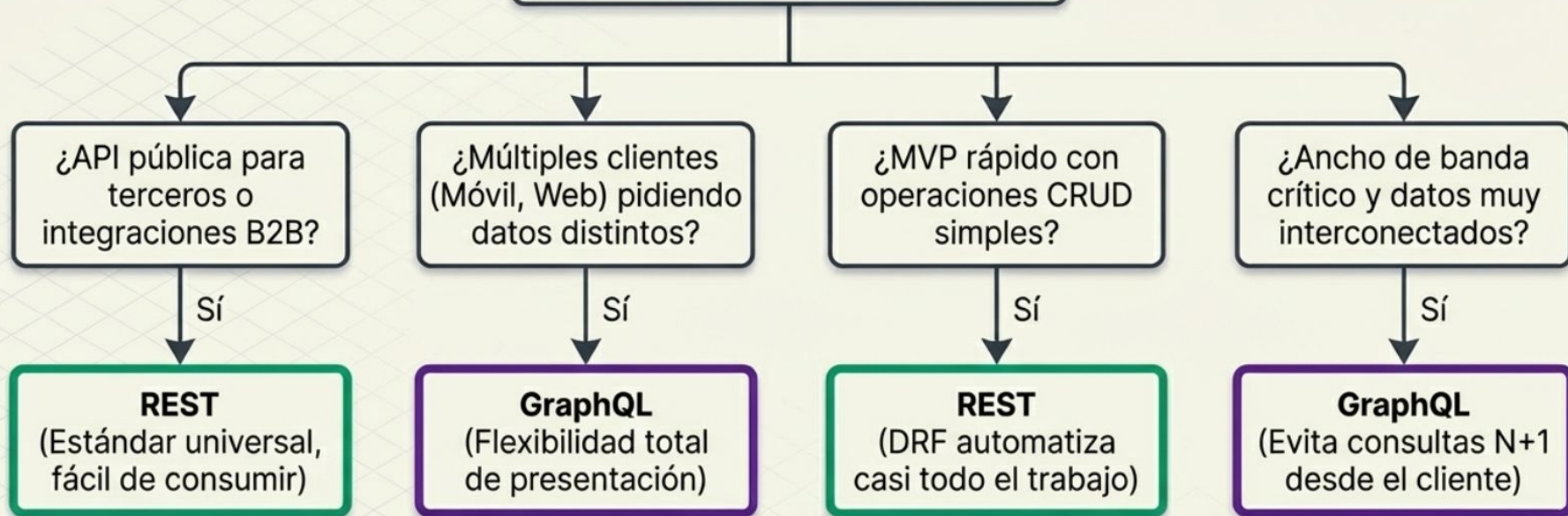
```
GraphiQL   
1 query ObtenerDashboardRepartidor {  
2   repartidor(id: 123) {  
3     nombre  
4     entregasPendientes {  
5       direccion  
6       estado  
7     }  
8   }  
9 }
```

```
{  
  "data": {  
    "repartidor": {  
      "nombre": "Carlos",  
      "entregasPendientes": [  
        {  
          "direccion": "Edificio A, Laboratorio 3",  
          "estado": "pendiente"  
        }  
      ]  
    }  
  }  
}
```

Magia Estructural: Ni un dato más, ni un dato menos.
El backend procesa exactamente lo requerido gracias a los Resolvers.

Árbol de Decisión del Arquitecto

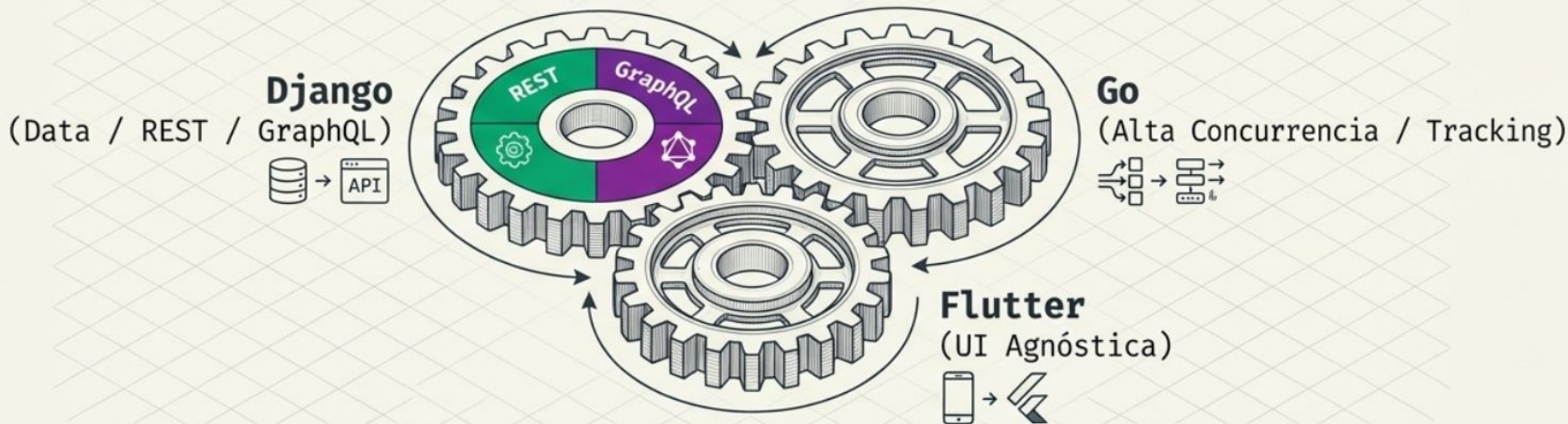
Árbol de Decisión del Arquitecto



⚡ Ambas herramientas coexisten. La arquitectura depende del dominio del problema.

Ingeniería de Software Integral

No hay una bala de plata en la arquitectura de software.



- **Django** proporciona un ecosistema inigualable para modelar reglas de negocio complejas.
- Dominar el desacoplamiento con DRF o GraphQL es la marca de un ingeniero moderno.
- Próximo paso: Integración de microservicios concurrentes (**Goroutines** en Go).